



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Informatyki

Projekt dyplomowy

Projekt i implementacja zintegrowanego systemu analizy danych medycznych z wykorzystaniem sztucznej inteligencji

Design and implementation of an integrated medical data analysis system using artificial intelligence

Autorzy pracy: Damian Chłus, Jakub Gucwa,
Gabriel Kania, Filip Malejki

Kierunek studiów: Informatyka

Opiekun pracy: dr inż. Anna Wójcicka

Spis treści

1	Wizja projektu	5
1.1	Opis dziedziny problemu	5
1.2	Rola projektowanego systemu i użytkownicy końcowi	6
1.3	Współpracujące systemy zewnętrzne	6
1.4	Obszary funkcjonalne i нефunkcjonalne	6
1.4.1	Wymagania funkcjonalne	6
1.4.2	Wymagania нефunkcjonalne	7
1.5	Przegląd istniejących rozwiązań	7
1.6	Architektura systemu	8
1.6.1	Środowisko uruchomieniowe i orkiestracja	8
1.6.2	Brama wejściowa (<i>Ingress</i>) i Warstwa Prezentacji (<i>Frontend</i>)	8
1.6.3	Warstwa Logiki Biznesowej	8
1.6.4	Warstwa Utrwalania Danych (<i>Baza Danych</i>) i Kopie Zapasowe	8
1.6.5	Moduł Decyzyjny (<i>Scheduler / Orkiestrator Modeli</i>)	8
1.6.6	Konteneryzowane Modele Uczenia Maszynowego (<i>ML</i>)	9
1.6.7	Infrastruktura jako Kod (<i>IaC</i>) i podejście GitOps	9
1.7	Studium wykonalności	11
1.8	Analiza ryzyka	12
1.9	Słownik pojęć	13
2	Specyfikacja wymagań	14
2.1	Charakterystyka użytkowników i systemów współpracujących	14
2.1.1	Lekarz	14
2.1.2	Administrator	14
2.1.3	Deweloper modułów	15
2.1.4	Moduł diagnostyczny	15
2.1.5	Systemy współpracujące	15
2.2	Funkcje systemu	15
2.2.1	Funkcje lekarza	16
2.2.2	Funkcje administratora	16
2.2.3	Funkcje dewelopera modułów	17
2.2.4	Funkcje platformy	17
2.3	Zakres projektu	18
2.4	Procesy systemowe	19
2.4.1	Proces analizy diagnostycznej	19
2.4.2	Proces integracji nowego modułu diagnostycznego	21
2.5	Scenariusze użytkowania i testowania	23
2.5.1	Przesłanie danych pacjenta do analizy	23
2.5.2	Analiza danych przez moduł diagnostyczny	23
2.5.3	Dodanie nowego modułu diagnostycznego	24
2.5.4	Scenariusze testowe	24
2.6	Koncepcja interfejsu użytkownika	26
2.6.1	Ekran logowania	26
2.6.2	Panel lekarza	27
2.6.3	Formularz przesyłania badań	27

2.6.4	Widok wyników analizy	27
2.6.5	Panel administratora	28
2.6.6	Założenia projektowe interfejsu	28
2.7	Specyfikacja API i komunikacji	28
2.7.1	API backendu	29
2.7.2	Komunikacja backendu z schedulerem	29
2.7.3	API modułów diagnostycznych	30
2.7.4	Manifest modułu diagnostycznego	30
2.7.5	Format danych	32
	Bibliografia	33

1 Wizja projektu

1.1 Opis dziedziny problemu

Współczesne systemy wspomaganie diagnostyki medycznej coraz częściej wykorzystują metody sztucznej inteligencji oraz uczenia maszynowego do analizy danych pacjentów. Rozwiązania tego typu znajdują zastosowanie między innymi w analizie obrazów medycznych, interpretacji wyników badań laboratoryjnych oraz wspomaganiu procesu podejmowania decyzji diagnostycznych przez lekarzy. W praktyce klinicznej systemy te mają najczęściej charakter wyspecjalizowanych narzędzi przeznaczonych do analizy pojedynczego rodzaju danych lub konkretnego obszaru medycznego.

Problem projektowy dotyczy stworzenia platformy umożliwiającej integrację wielu wyspecjalizowanych modułów diagnostycznych w ramach jednego systemu. Projekt zakłada wykorzystanie podejścia inspirowanego techniką *Mixture of Experts*, w którym niezależne moduły uczenia maszynowego odpowiadają za analizę konkretnych typów danych medycznych lub określonych obszarów diagnostycznych.

W ramach projektu planowane jest dostarczenie kompletnej platformy diagnostycznej wraz z dwoma przykładowymi modułami wykorzystującymi modele uczenia maszynowego. Moduły te mają wspomagać analizę przypadków związanych z **miażdżycą** poprzez interpretację **badania przesiewowego** oraz analizę **angiogramów naczyń wieńcowych**. Projekt obejmuje również opracowanie wytycznych oraz specyfikacji umożliwiającej tworzenie i integrację kolejnych modułów diagnostycznych przez zewnętrznych deweloperów.

Głównymi założeniami projektu są:

- Dostarczenie modularnej platformy wspomaganie diagnostycznego
- Implementacja przykładowych modułów diagnostycznych opartych na modelach uczenia maszynowego
- Umożliwienie dalszej rozbudowy systemu o kolejne moduły
- Określenie wytycznych dotyczących tworzenia i integracji nowych modułów
- Stworzenie niezawodnej architektury systemu
- Zapewnienie bezpieczeństwa przetwarzanych danych

Docelowym odbiorcą projektowanego oprogramowania są **placówki szpitalne** oraz personel medyczny wspomagający proces diagnostyczny. Charakter przetwarzanych informacji wymaga zapewnienia wysokiego poziomu bezpieczeństwa danych, kontroli dostępu oraz niezawodności działania systemu. Ze względu na potencjalne zastosowanie w środowiskach o podwyższonych wymaganiach bezpieczeństwa projekt zakłada możliwość działania systemu w infrastrukturze lokalnej, bez konieczności korzystania z usług zewnętrznych lub środowisk chmurowych.

1.2 Rola projektowanego systemu i użytkownicy końcowi

Projektowany system stanowi narzędzie wspomagające proces diagnostyczny poprzez analizę danych medycznych z wykorzystaniem **wyspecjalizowanych modułów** uczenia maszynowego. Jego głównym zadaniem jest dostarczanie personelowi medycznemu sugestii diagnostycznych na podstawie dostarczonych wyników badań, wspierając tym samym proces podejmowania decyzji przez lekarza.

System **nie pełni funkcji autonomicznego narzędzia diagnostycznego** i nie zastępuje specjalisty medycznego, a jedynie stanowi wsparcie informacyjne podczas analizy przypadków pacjentów z podejrzeniem schorzeń objętych zakresem działania zaimplementowanych modułów.

Głównymi użytkownikami końcowymi systemu są lekarze oraz inny uprawniony personel medyczny posiadający kompetencje do interpretacji wyników diagnostycznych i podejmowania decyzji klinicznych.

Dodatkowo w ekosystemie systemu wyróżnia się role administracyjne i techniczne, obejmujące administratorów odpowiedzialnych za utrzymanie infrastruktury i zarządzanie kontami użytkowników oraz deweloperów odpowiedzialnych za rozwój i integrację nowych modułów diagnostycznych zgodnie z przyjętymi standardami architektonicznymi.

1.3 Współpracujące systemy zewnętrzne

W naszym projekcie priorytetem jest **samowystarczalność** i **niezawodność** systemu informatycznego, który w skrajnych przypadkach jest w stanie funkcjonować bez połączenia z siecią internet. W związku z powyższym nie zakładamy użycia systemów zewnętrznych.

1.4 Obszary funkcjonalne i нефunkcjonalne

Wymagania projektowe zostały podzielone na dwie główne kategorie: wymagania **funkcjonalne** oraz wymagania **niefunkcjonalne**. Pierwsza grupa określa zakres operacji realizowanych przez system, natomiast druga definiuje oczekiwane cechy jakościowe oraz ograniczenia techniczne projektowanego rozwiązania.

1.4.1 Wymagania funkcjonalne

- Administrator posiada możliwość tworzenia oraz zarządzania kontami użytkowników systemu,
- Administrator posiada możliwość zarządzania dostępnymi modułami diagnostycznymi,
- Użytkownik posiada możliwość uwierzytelnienia w systemie,
- Użytkownik może przysyłać dane pacjenta w wymaganym formacie oraz otrzymywać wyniki analizy w czytelnej formie,
- System umożliwia analizę danych pochodzących z różnych źródeł diagnostycznych,
- System umożliwia wykorzystanie przykładowych modułów diagnostycznych dostarczonych w ramach projektu,
- System umożliwia integrację dodatkowych modułów diagnostycznych tworzonych przez podmioty trzecie,
- Deweloper modułów posiada możliwość rozszerzania systemu o nowe moduły diagnostyczne zgodne z przygotowaną specyfikacją integracyjną.

1.4.2 Wymagania niefunkcjonalne

- System powinien być możliwy do wdrożenia lokalnie na infrastrukturze placówki medycznej,
- System powinien umożliwiać działanie w środowisku odizolowanym od sieci publicznej,
- Architektura systemu powinna umożliwiać integrację nowych modułów bez konieczności przebudowy istniejących komponentów,
- Projekt powinien zapewniać specyfikację integracyjną opisującą sposób implementacji oraz dołączania nowych modułów diagnostycznych,
- Poszczególne komponenty systemu powinny być możliwe do niezależnego wdrażania i rozwijania.

1.5 Przegląd istniejących rozwiązań

Na rynku dostępnych jest wiele systemów wspomagających proces diagnostyczny, jednak większość z nich skupia się na pojedynczym obszarze zastosowań lub jednym typie danych wejściowych.

Znaczna część obecnych rozwiązań działa w oparciu o analizę symptomów oraz danych opisowych przekazywanych przez użytkownika w celu wygenerowania listy potencjalnych przyczyn lub sugestii diagnostycznych. Przykładem takiego rozwiązania jest **Isabel DDx Companion**, wspierający lekarzy poprzez analizę objawów oraz historii medycznej pacjenta.

Istnieją również rozwiązania wyspecjalizowane w analizie konkretnych typów badań diagnostycznych. Przykładowo:

- **Annalise.ai** specjalizuje się w analizie obrazów radiologicznych, szczególnie zdjęć RTG,
- **Google DermAssist** wspomaga analizę zmian skórnych na podstawie zdjęć dermatologicznych,
- **Aidoc** oferuje narzędzia do analizy obrazów tomografii komputerowej i rezonansu magnetycznego.

W ostatnich latach rozwijane są również bardziej zaawansowane systemy eksperymentalne wykorzystujące modele językowe i podejście multimodalne. Jednym z najważniejszych przykładów jest rozwijany przez Google system **AMIE (Articulate Medical Intelligence Explorer)**^[1], będący badawczym agentem diagnostycznym zdolnym do prowadzenia dialogu medycznego oraz analizy danych tekstowych i wizualnych.

Kolejnym przykładem nowoczesnego podejścia do diagnostyki wspomaganej sztuczną inteligencją jest **Med-Gemini**^{[2], [3]} – multimodalny model językowy rozwijany przez Google, przystosowany do analizy danych medycznych obejmujących tekst, obrazy oraz dokumentację kliniczną. Rozwiązanie to reprezentuje aktualny kierunek rozwoju branży, zmierzający w stronę uniwersalnych modeli zdolnych do jednoczesnego przetwarzania wielu źródeł informacji medycznych.

Pomimo wysokiego poziomu zaawansowania istniejących rozwiązań, wiele z nich pozostaje ograniczonych do pojedynczej specjalizacji lub opiera się na jednym scentralizowanym modelu sztucznej inteligencji, co może utrudniać dalszą specjalizację lub elastyczne rozszerzanie systemu.

Projektowane rozwiązanie wyróżnia się poprzez:

- modularną architekturę umożliwiającą integrację wielu wyspecjalizowanych modułów diagnostycznych,
- możliwość jednoczesnego przetwarzania różnych typów danych wejściowych,
- łatwość rozbudowy o nowe moduły bez konieczności przebudowy całego systemu,
- elastyczność pozwalającą dostosować system do nowych przypadków użycia.

1.6 Architektura systemu

1.6.1 Środowisko uruchomieniowe i orkiestracja

Architektura systemu opiera się na *klastrze K3s* (odchudzonej dystrybucji systemu Kubernetes), zaprojektowanym do działania w środowisku **izolowanym** (air-gapped / edge computing). Ze względu na specyfikację i skalę projektu, środowisko zostało wdrożone na pojedynczym węźle fizycznym (serwerze). W projekcie świadomie zaakceptowano występowanie pojedynczego punktu awarii (*SPOF* – *Single Point of Failure*) na poziomie sprzętowym. K3s wykorzystywany jest tu nie w celu zapewnienia wysokiej dostępności sprzętowej (*HA*), lecz w celu orkiestracji kontenerów, automatyzacji wdrożeń, izolacji procesów oraz zapewnienia niezawodności na poziomie aplikacyjnym.

1.6.2 Brama wejściowa (*Ingress*) i Warstwa Prezentacji (*Frontend*)

Ruch sieciowy do klastra zarządzany jest przez kontroler *Ingress* (np. Traefik lub Nginx Ingress), który pełni rolę **odwrotnego serwera proxy** (Reverse Proxy) i kieruje zapytania do odpowiednich usług.

Warstwa prezentacji oparta jest na frameworku *Next.js*, uruchomionym w dedykowanych kontenerach ze środowiskiem uruchomieniowym *Node.js* (lub serwowanym jako statyczne pliki przez serwer Nginx, w zależności od wybranego trybu budowania aplikacji). Aplikacja frontendowa jest wdrożona z użyciem obiektu **Deployment w 2 replikach**, do których ruch rozdzielany jest równomiernie za pomocą obiektu **Service** (wewnętrzny load balancing).

1.6.3 Warstwa Logiki Biznesowej

Główny interfejs programistyczny (API) oraz logika biznesowa obsługiwane są przez aplikację napisaną w frameworku *Django*. Backend został wdrożony jako oddzielny **Deployment** (2 repliki, zarządzane przez **Service** z load balancingiem). Aplikacja ta komunikuje się z warstwą prezentacji (odbierając żądania HTTP z frontendu), zarządza połączeniem z relacyjną bazą danych oraz asynchronicznie odpytuje moduł decyzyjny (Scheduler).

1.6.4 Warstwa Utrwalania Danych (*Baza Danych*) i Kopie Zapasowe

Jako główny magazyn danych relacyjnych wykorzystano system *PostgreSQL*. W celu zapewnienia trwałości danych po restarcie kontenerów, baza została wdrożona z wykorzystaniem kontrolera **StatefulSet** oraz powiązana z wolumenem trwałym (*PVC* – *Persistent Volume Claim*). Ponieważ *PVC* zapewnia jedynie trwałość, a nie ochronę przed logicznym uszkodzeniem danych, architektura zakłada istnienie dedykowanego **zadania cyklicznego** (np. obiekt *CronJob* w K3s), które regularnie wykonuje zrzuty bazy danych i zapisuje je w wydzielonej przestrzeni, pełniąc funkcję pełnoprawnej kopii zapasowej (Backup). Baza danych jest odizolowana i przyjmuje połączenia **wyłącznie od podów backendu Django**.

1.6.5 Moduł Decyzyjny (*Scheduler* / *Orkiestrator Modeli*)

Pełni rolę inteligentnego węzła sterującego przepływem danych analitycznych. Jego działanie opiera się na koncepcji orkiestracji dziedzicznej. Scheduler analizuje metadane opisujące zakres kompetencji każdego dostępnego modułu ML (np. „badania przesiewowe krwi”, „obrazowanie angiograficzne”). Na podstawie kontekstu zapytania, moduł ten podejmuje autonomiczną decyzję

o selekcji odpowiedniego zestawu modeli niezbędnych do rozwiązania danego problemu diagnostycznego. Kluczowym zadaniem Schedulera jest fuzja wyników (ang. data fusion): orkiestrator integruje rozproszone dane wyjściowe z wybranych podów (np. łącząc parametry hematologiczne z wynikami analizy obrazowej), a następnie przeprowadza proces wnioskowania wyższego rzędu. Wynikiem działania tego modułu jest spójna, wielowymiarowa synteza, stanowiąca konkretny wniosek końcowy oparty na komplementarnych źródłach informacji.

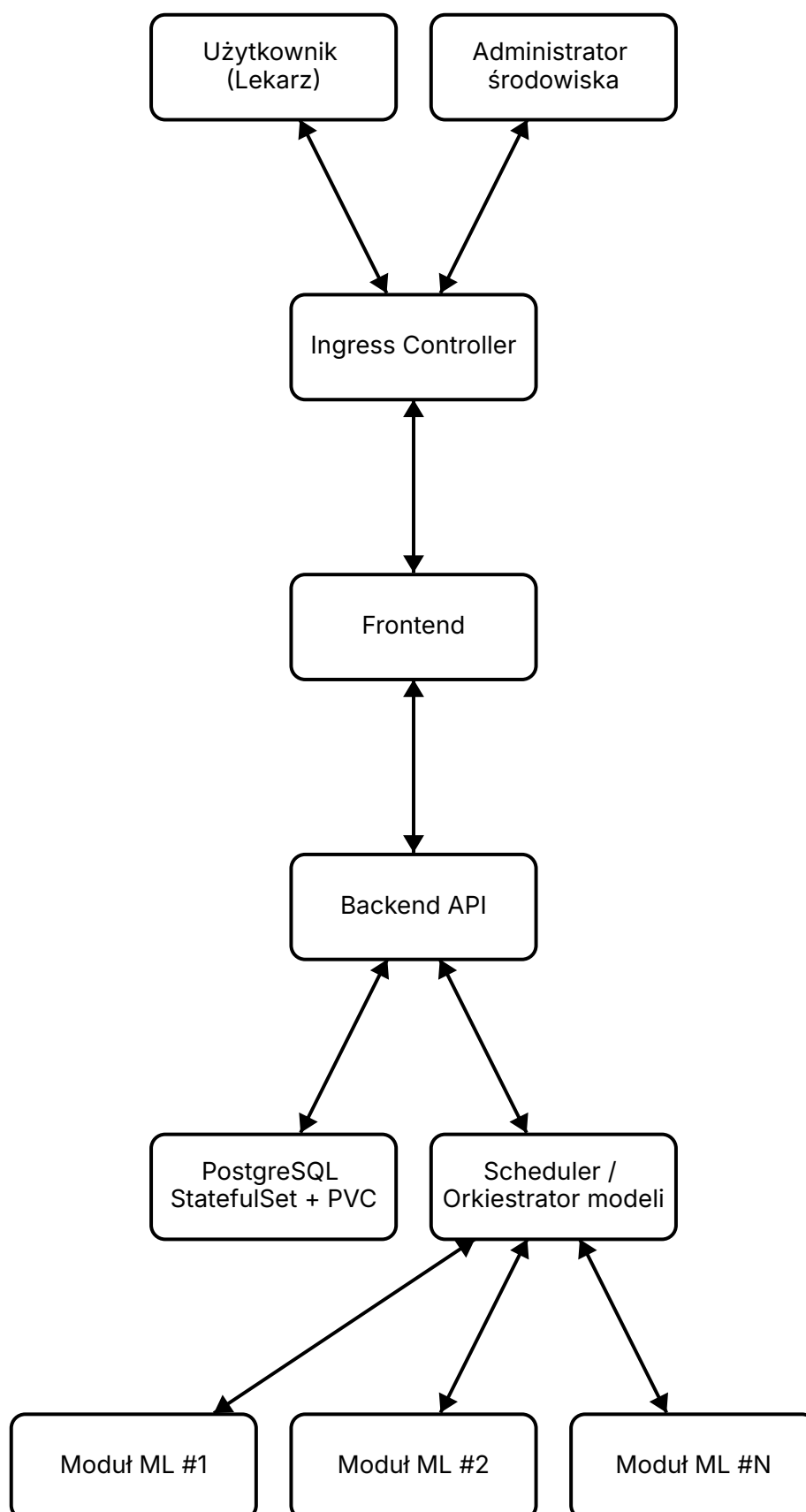
1.6.6 Konteneryzowane Modele Uczenia Maszynowego (ML)

Każdy wytrenowany do konkretnego celu model ML jest wdrożony jako niezależny **mikroserwis** (własny **Deployment** oraz **Service**), udostępniając **własne API** (np. oparte na frameworku FastAPI). Ze względu na duże zapotrzebowanie na moc obliczeniową, pody z modelami posiadają ściśle zdefiniowane limity zasobów (`resources.requests` i `resources.limits` dla CPU/RAM lub GPU), co zapobiega wyczerpaniu zasobów całego serwera. Skalowanie poziome poszczególnych modeli oraz load balancing realizowane są natywnie przez mechanizmy K3s.

1.6.7 Infrastruktura jako Kod (IaC) i podejście GitOps

Zarządzanie architekturą modeli ML odbywa się zgodnie z paradygmatem *Infrastructure as Code*. Wszystkie konfiguracje i definicje usług opisane są deklaratywnie w plikach YAML. Wdrożono narzędzie klasy **GitOps** (np. Argo CD), które stale monitoruje dedykowane repozytorium konfiguracyjne. Wprowadzenie zmian w plikach w repozytorium (np. dodanie pliku YAML dla nowego modelu) skutkuje **automatyczną synchronizacją i przebudową** infrastruktury w klastrze K3s, eliminując konieczność manualnej ingerencji w środowisko produkcyjne.

Graf 1: Schemat blokowy architektury



1.7 Studium wykonalności

Realizacja projektu jest możliwa dzięki szerokiej dostępności nowoczesnych narzędzi wspierających rozwój aplikacji webowych, systemów backendowych, modeli uczenia maszynowego oraz platform do orkiestracji konteneryzowanych aplikacji. Aktualny stan technologii umożliwia budowę rozproszonych i modularnych systemów zdolnych do integracji wielu niezależnych komponentów analitycznych w ramach jednej platformy.

Z perspektywy architektonicznej projekt opiera się na podejściu inspirowanym techniką *Mixture of Experts*, polegającej na wykorzystaniu wielu wyspecjalizowanych modułów odpowiedzialnych za analizę konkretnych typów danych lub obszarów diagnostycznych. Zastosowanie takiego modelu umożliwia logiczne rozdzielenie odpowiedzialności pomiędzy komponenty systemu, co upraszcza proces implementacji, testowania oraz dalszego rozwoju projektu.

Każdy moduł diagnostyczny może być rozwijany oraz wdrażany niezależnie, bez konieczności ingerencji w pozostałe elementy systemu, co znacząco zwiększa elastyczność rozwiązania oraz umożliwia jego dalszą rozbudowę wraz z pojawianiem się nowych potrzeb diagnostycznych.

Istotnym aspektem wykonalności projektu jest możliwość wdrożenia systemu w środowisku lokalnym, bez konieczności wykorzystania infrastruktury chmurowej. Dzięki zastosowaniu konteneryzacji oraz narzędzi orkiestracyjnych możliwe jest uruchomienie całej platformy na dedykowanej infrastrukturze placówki medycznej przy zachowaniu izolacji środowiska oraz wysokiego poziomu kontroli nad przetwarzanymi danymi.

Wykonalność projektu potwierdza również dostępność gotowych bibliotek, frameworków oraz modeli referencyjnych umożliwiających implementację algorytmów analizy danych tekstowych i obrazowych. Pozwala to skupić proces projektowy na integracji komponentów oraz dostosowaniu modeli do konkretnych zastosowań medycznych zamiast budowy wszystkich rozwiązań od podstaw.

Dodatkowym czynnikiem wpływającym na wykonalność projektu jest możliwość etapowego rozwoju systemu. Modularna architektura umożliwia rozpoczęcie prac od ograniczonego zestawu funkcjonalności i sukcesywne rozszerzanie systemu o kolejne moduły diagnostyczne w przyszłości.

Zespół projektowy posiada doświadczenie w zakresie tworzenia aplikacji webowych, systemów backendowych, konteneryzacji aplikacji oraz projektowania architektur opartych o mikroserwisy. Członkowie zespołu posiadają również praktyczną wiedzę z zakresu integracji systemów, pracy z narzędziami orkiestracji kontenerów oraz implementacji modeli uczenia maszynowego wykorzystywanych w przykładowych modułach diagnostycznych.

Przeprowadzone analizy technologiczne oraz wstępne prace koncepcyjne potwierdzają możliwość realizacji założeń projektowych przy wykorzystaniu obecnie dostępnych technologii i metod inżynierii oprogramowania.

1.8 Analiza ryzyka

Ze względu na modularny charakter projektu oraz podział prac pomiędzy niezależne komponenty istnieje ryzyko wystąpienia problemów integracyjnych podczas łączenia poszczególnych elementów systemu w jedną spójną całość. Ryzyko to obejmuje możliwość wystąpienia błędów komunikacyjnych pomiędzy modułami, niezgodności interfejsów oraz opóźnień wynikających z konieczności dostosowywania komponentów tworzonych równolegle przez różnych członków zespołu. Minimalizacja tego ryzyka realizowana jest poprzez wcześniejsze ustalenie wspólnych standardów komunikacji oraz interfejsów pomiędzy modułami.

Istotnym zagrożeniem projektowym pozostaje również ryzyko związane z procesem przygotowania oraz trenowania modeli uczenia maszynowego wykorzystywanych w przykładowych modułach diagnostycznych. W szczególności istnieje możliwość nieprawidłowego przygotowania modeli skutkującego zjawiskami takimi jak przetrenowanie, niedouczenie lub niewystarczająca generalizacja na nowych danych. Ograniczenie tego ryzyka realizowane jest poprzez testowanie modeli na wydzielonych zbiorach walidacyjnych oraz iteracyjne dostosowywanie parametrów procesu uczenia.

Dodatkowym ryzykiem jest ograniczona dostępność odpowiednich danych treningowych lub niewystarczająca jakość dostępnych zbiorów danych, co może negatywnie wpłynąć na skuteczność działania modułów diagnostycznych. W celu ograniczenia tego zagrożenia planowane jest wykorzystanie publicznie dostępnych zbiorów referencyjnych oraz analiza jakości danych przed rozpoczęciem procesu trenowania.

Ze względu na zastosowanie architektury mikroserwisowej oraz orkiestracji kontenerów istnieje również ryzyko zwiększonej złożoności infrastrukturalnej systemu. Obejmuje ono możliwość wystąpienia problemów konfiguracyjnych, trudności we wdrażaniu oraz błędów wynikających z nieprawidłowej konfiguracji środowiska uruchomieniowego. Ryzyko to ograniczane jest poprzez stosowanie podejścia Infrastructure as Code oraz automatyzację procesu wdrożeniowego.

Ryzyko projektowe obejmuje także możliwość wystąpienia ograniczeń sprzętowych lub wydajnościowych podczas trenowania modeli oraz testowania całego systemu. Ograniczenie tego ryzyka realizowane jest poprzez etapowe wdrażanie funkcjonalności, bieżące monitorowanie zużycia zasobów oraz testowanie wydajności implementowanych komponentów.

Dodatkowym wyzwaniem projektowym jest opracowanie uniwersalnego oraz spójnego sposobu integracji nowych modułów diagnostycznych z platformą. Mechanizm ten powinien być jednocześnie prosty w implementacji, zrozumiały dla deweloperów tworzących nowe moduły oraz elastyczny względem różnych rodzajów przetwarzanych danych, takich jak dane tekstowe i obrazowe.

1.9 Słownik pojęć

- **Moduł diagnostyczny** – niezależny komponent systemu odpowiedzialny za analizę określonego typu danych medycznych.
- **Mixture of Experts (MoE)** – podejście architektoniczne polegające na wykorzystaniu wielu wyspecjalizowanych modułów eksperckich przeznaczonych do realizacji konkretnych zadań.
- **Badania przesiewowe** – podstawowe badania diagnostyczne wykonywane w celu wczesnego wykrywania potencjalnych nieprawidłowości zdrowotnych.
- **Angiogram naczyń wieńcowych** – obraz diagnostyczny przedstawiający naczynia krwionośne serca, wykorzystywany w ocenie ich drożności i ewentualnych zmian chorobowych.
- **Modularność systemu** – właściwość architektury umożliwiająca rozwój oraz modyfikację poszczególnych komponentów bez wpływu na całość rozwiązania.
- **Mikroserwis** – niezależnie wdrażalny komponent aplikacji realizujący wydzieloną część funkcjonalności systemu.
- **Scheduler** – komponent odpowiedzialny za kierowanie zapytań do odpowiednich modułów diagnostycznych na podstawie rodzaju przetwarzanych danych.
- **Konteneryzacja** – metoda uruchamiania aplikacji w izolowanych środowiskach zawierających wszystkie wymagane zależności.
- **Orkiestracja kontenerów** – proces automatycznego zarządzania wdrażaniem, skalowaniem i utrzymaniem konteneryzowanych aplikacji.
- **Kubernetes / K3s** – platforma służąca do orkiestracji kontenerów; K3s stanowi jej uproszczoną i odchudzoną wersję.
- **Ingress** – mechanizm zarządzający ruchem sieciowym kierowanym do usług działających wewnątrz klastra.
- **GitOps** – podejście do zarządzania infrastrukturą, w którym konfiguracja środowiska przechowywana jest w repozytorium kodu i automatycznie synchronizowana z systemem docelowym.

2 Specyfikacja wymagań

Specyfikacja wymagań stanowi formalny opis funkcjonalności oraz założeń projektowanego systemu wspomagania diagnostycznego. Celem tej części dokumentacji jest doprecyzowanie sposobu działania platformy, zakresu odpowiedzialności poszczególnych komponentów oraz interakcji pomiędzy użytkownikami a systemem.

W kolejnych sekcjach przedstawiono charakterystykę aktorów systemu, szczegółowy opis funkcji platformy, zakres realizowanego projektu, scenariusze użytkowania oraz koncepcję komunikacji pomiędzy komponentami systemu i modułami diagnostycznymi.

2.1 Charakterystyka użytkowników i systemów współpracujących

Projektowany system wykorzystuje architekturę modułową, w której poszczególne elementy platformy realizują określone zadania związane z analizą danych medycznych, zarządzaniem użytkownikami oraz integracją modułów diagnostycznych. W ramach systemu wyróżniono formalnych aktorów odpowiedzialnych za korzystanie z platformy oraz systemy współpracujące komunikujące się z jej komponentami.

Aktor / System	Typ	Opis
Lekarz	Użytkownik	Personel medyczny korzystający z systemu w celu przesyłania danych diagnostycznych oraz przeglądania wyników analizy przygotowanych przez moduły diagnostyczne.
Administrator	Użytkownik	Osoba odpowiedzialna za zarządzanie użytkownikami systemu, monitorowanie działania platformy oraz integrację nowych modułów diagnostycznych.
Deweloper modułów	Interesariusz zewnętrzny	Osoba lub zespół odpowiedzialny za implementację niezależnych modułów diagnostycznych zgodnych ze specyfikacją platformy.
Moduł diagnostyczny	System współpracujący	Niezależny mikroservis realizujący analizę określonego typu danych medycznych z wykorzystaniem modeli uczenia maszynowego.

2.1.1 Lekarz

Lekarz stanowi głównego użytkownika funkcjonalnego systemu. Jego zadaniem jest dostarczanie danych diagnostycznych pacjenta do platformy oraz interpretacja otrzymanych wyników analizy wspomagających proces diagnostyczny. Lekarz posiada dostęp wyłącznie do funkcji związanych z analizą danych medycznych oraz historią wykonanych analiz.

2.1.2 Administrator

Administrator odpowiada za zarządzanie kontami użytkowników oraz nadzorowanie działania platformy. Do jego obowiązków należy również integracja nowych modułów diagnostycznych zgodnych ze specyfikacją systemu. Administrator nie zarządza bezpośrednio infrastrukturą uruchomieniową platformy, jednak posiada możliwość monitorowania działania systemu oraz kontrolowania dostępnych modułów.

2.1.3 Deweloper modułów

Deweloper modułów jest zewnętrznym interesariuszem odpowiedzialnym za tworzenie nowych modułów diagnostycznych zgodnych z wymaganiami integracyjnymi platformy. Moduły implementowane przez deweloperów funkcjonują jako niezależne mikroserwisy komunikujące się z platformą za pomocą ustalonego interfejsu API.

2.1.4 Moduł diagnostyczny

Moduł diagnostyczny stanowi niezależny system współpracujący z platformą. Każdy moduł odpowiada za analizę określonego rodzaju danych medycznych oraz generowanie wyników diagnostycznych przekazywanych do platformy. Moduły mogą wykorzystywać różne modele uczenia maszynowego oraz różne typy danych wejściowych, jednak muszą implementować wspólny interfejs komunikacyjny wymagany przez platformę.

2.1.5 Systemy współpracujące

Platforma współpracuje z szeregiem wewnętrznych oraz zewnętrznych komponentów technicznych odpowiedzialnych za realizację funkcji diagnostycznych, przechowywanie danych oraz zarządzanie środowiskiem uruchomieniowym systemu.

System / Komponent	Opis
Moduły diagnostyczne ML	Niezależne mikroserwisy realizujące analizę danych medycznych oraz generowanie wyników diagnostycznych na podstawie modeli uczenia maszynowego.
Scheduler / Orkiestrator modeli	Wewnętrzny komponent platformy odpowiedzialny za kierowanie zapytań do odpowiednich modułów diagnostycznych oraz zarządzanie komunikacją pomiędzy komponentami systemu.
Relacyjna baza danych	Komponent odpowiedzialny za przechowywanie danych aplikacyjnych, informacji o użytkownikach oraz historii analiz wykonywanych przez platformę.
Środowisko orkiestracji kontenerów	Warstwa infrastrukturalna odpowiedzialna za uruchamianie, skalowanie oraz monitorowanie kontenerów i mikroservisów tworzących platformę.
System GitOps	Komponent odpowiedzialny za automatyzację wdrożeń oraz synchronizację konfiguracji infrastruktury na podstawie repozytorium konfiguracyjnego.

Na obecnym etapie projekt nie zakłada integracji z zewnętrznymi systemami szpitalnymi ani usługami chmurowymi. Platforma projektowana jest jako rozwiązanie autonomiczne działające w środowisku lokalnym.

2.2 Funkcje systemu

Projektowany system realizuje funkcje związane z analizą danych medycznych, zarządzaniem użytkownikami oraz integracją wyspecjalizowanych modułów diagnostycznych. Funkcjonalności zostały podzielone według aktorów systemu oraz komponentów platformy. Dla poszczególnych funkcji określono ich podstawowy zakres oraz priorytet zgodnie z metodą MoSCoW.

2.2.1 Funkcje lekarza

Funkcja	Opis	Priorytet
Logowanie do systemu	Uwierzytelnienie użytkownika oraz uzyskanie dostępu do funkcji platformy zgodnych z posiadanymi uprawnieniami.	Must Have
Wybór rodzaju analizy	Wybór obszaru diagnostycznego, względem którego mają zostać przeanalizowane dostarczone dane medyczne.	Must Have
Przesyłanie danych diagnostycznych	Przekazywanie anonimowych danych diagnostycznych, takich jak wyniki badań laboratoryjnych lub obrazy medyczne, do platformy.	Must Have
Uruchomienie procesu analizy	Zainicjowanie procesu analizy danych przez platformę oraz odpowiednie moduły diagnostyczne.	Must Have
Przegląd wyników analizy	Wyświetlenie wyników analizy wygenerowanych przez moduły diagnostyczne w czytelnej formie.	Must Have

Lekarz nie posiada dostępu do konfiguracji systemu ani infrastruktury platformy. System nie przechowuje historii pacjentów ani trwałych profili medycznych – dane przekazywane są jednorazowo na potrzeby wykonania analizy.

2.2.2 Funkcje administratora

Funkcja	Opis	Priorytet
Zarządzanie użytkownikami	Tworzenie, edycja oraz dezaktywacja kont użytkowników systemu.	Must Have
Zarządzanie uprawnieniami	Kontrola dostępu użytkowników do funkcji platformy.	Must Have
Integracja modułów diagnostycznych	Dodawanie nowych modułów diagnostycznych poprzez dostarczenie manifestu konfiguracyjnego opisującego moduł oraz jego interfejs komunikacyjny.	Must Have
Monitoring działania systemu	Podstawowy podgląd stanu komponentów platformy oraz dostępności modułów diagnostycznych.	Should Have

Administrator nie zarządza bezpośrednio infrastrukturą uruchomieniową platformy, jednak posiada możliwość nadzorowania działania systemu oraz kontrolowania dostępnych modułów diagnostycznych.

2.2.3 Funkcje dewelopera modułów

Funkcja	Opis	Priorytet
Implementacja modułu diagnostycznego	Tworzenie niezależnego mikroserwisu realizującego analizę określonego typu danych medycznych.	Must Have
Implementacja interfejsu API	Przygotowanie interfejsu komunikacyjnego zgodnego ze specyfikacją platformy.	Must Have
Przygotowanie manifestu modułu	Dostarczenie manifestu opisującego konfigurację, wymagania oraz sposób komunikacji modułu z platformą.	Must Have
Przygotowanie modułu do wdrożenia	Dostarczenie gotowego do uruchomienia kontenera lub zestawu plików wdrożeniowych.	Should Have

Deweloper modułów działa poza platformą i nie korzysta bezpośrednio z jej interfejsu użytkownika. Integracja modułu odbywa się poprzez zgodność ze specyfikacją komunikacyjną systemu.

2.2.4 Funkcje platformy

Funkcja	Opis	Priorytet
Routing zapytań diagnostycznych	Przekazywanie żądań do odpowiednich modułów diagnostycznych na podstawie rodzaju analizy oraz typu danych wejściowych.	Must Have
Komunikacja pomiędzy komponentami	Obsługa komunikacji pomiędzy frontendem, backendem, schedulerem oraz modułami diagnostycznymi.	Must Have
Autoryzacja i kontrola dostępu	Weryfikacja uprawnień użytkowników oraz zabezpieczenie dostępu do funkcji systemu.	Must Have
Obsługa błędów i timeoutów	Wykrywanie błędów komunikacyjnych, timeoutów oraz obsługa sytuacji awaryjnych podczas analizy danych.	Should Have
Mechanizmy wysokiej dostępności	Wspieranie redundancji usług, automatycznego restartowania komponentów oraz równoważenia obciążenia.	Should Have
Logowanie zdarzeń systemowych	Rejestrowanie zdarzeń administracyjnych oraz technicznych związanych z działaniem platformy.	Should Have

Platforma odpowiada za koordynację komunikacji pomiędzy komponentami systemu oraz ukrywa przed użytkownikiem szczegóły działania infrastruktury i modułów diagnostycznych.

2.3 Zakres projektu

Zakres projektu obejmuje stworzenie modularnej platformy wspomagania diagnostycznego wraz z referencyjnymi modułami diagnostycznymi. Głównym celem jest dostarczenie działającego systemu demonstracyjnego, który pokazuje możliwość integracji różnych typów danych medycznych oraz rozbudowy platformy o kolejne moduły zgodne z przyjętą specyfikacją.

Element	Status	Komentarz
Platforma wspomagania diagnostycznego	W zakresie	Implementacja głównych komponentów systemu, w tym frontendu, backendu, schedulera oraz mechanizmów komunikacji z modułami diagnostycznymi.
Moduł diagnostyczny badań przesiewowych	W zakresie	Referencyjny moduł analizujący dane tekstowe/liczbowe pochodzące z badań krwi w kontekście wykrywania miażdżycy.
Moduł diagnostyczny angiogramu naczyń wieńcowych	W zakresie	Referencyjny moduł obrazowy analizujący angiogramy naczyń wieńcowych pod kątem zmian związanych z miażdżycą.
Specyfikacja integracji modułów	W zakresie	Opracowanie zasad tworzenia i dołączania nowych modułów diagnostycznych poprzez manifest konfiguracyjny oraz zgodny interfejs komunikacyjny.
Interfejs użytkownika	W zakresie	Implementacja interfejsu dla lekarza i administratora, umożliwiającego korzystanie z podstawowych funkcji platformy.
Referencyjne wdrożenie lokalne	W zakresie	Przygotowanie środowiska odzwierciedlającego docelowe wdrożenie lokalne typu on-premise.
Integracja kolejnych modułów diagnostycznych	Planowane przyszłościowo	Platforma będzie przygotowana do rozszerzania, jednak implementacja dodatkowych modułów poza dwoma referencyjnymi nie należy do podstawowego zakresu projektu.
Zaawansowany monitoring i obserwowalność	Planowane przyszłościowo	System może zostać rozszerzony o bardziej rozbudowane mechanizmy monitoringu, metryk oraz wizualizacji stanu komponentów.
Integracja z systemami szpitalnymi	Poza zakresem	Projekt nie obejmuje integracji z systemami HIS, PACS, RIS, HL7/FHIR ani innymi istniejącymi systemami placówek medycznych.
Wdrożenie chmurowe	Poza zakresem	System projektowany jest z myślą o środowisku lokalnym, dlatego wdrożenie w publicznej chmurze nie należy do zakresu projektu.

Element	Status	Komentarz
Certyfikacja medyczna i wdrożenie kliniczne	Poza zakresem	Projekt ma charakter inżynierski i demonstracyjno-badawczy. System nie jest przeznaczony do samodzielnego podejmowania decyzji medycznych ani do użycia jako certyfikowane narzędzie kliniczne.
Automatyczne ponowne trenowanie modeli	Poza zakresem	Projekt nie zakłada implementacji mechanizmów ciągłego uczenia, automatycznego retrainingu ani federacyjnego uczenia modeli.

Modele diagnostyczne tworzone w ramach projektu mają charakter referencyjny i demonstracyjno-badawczy. Ich zadaniem jest pokazanie możliwości platformy oraz sposobu integracji modułów opartych na uczeniu maszynowym. Skuteczność modeli będzie zależna od dostępności i jakości wykorzystanych zbiorów danych.

2.4 Procesy systemowe

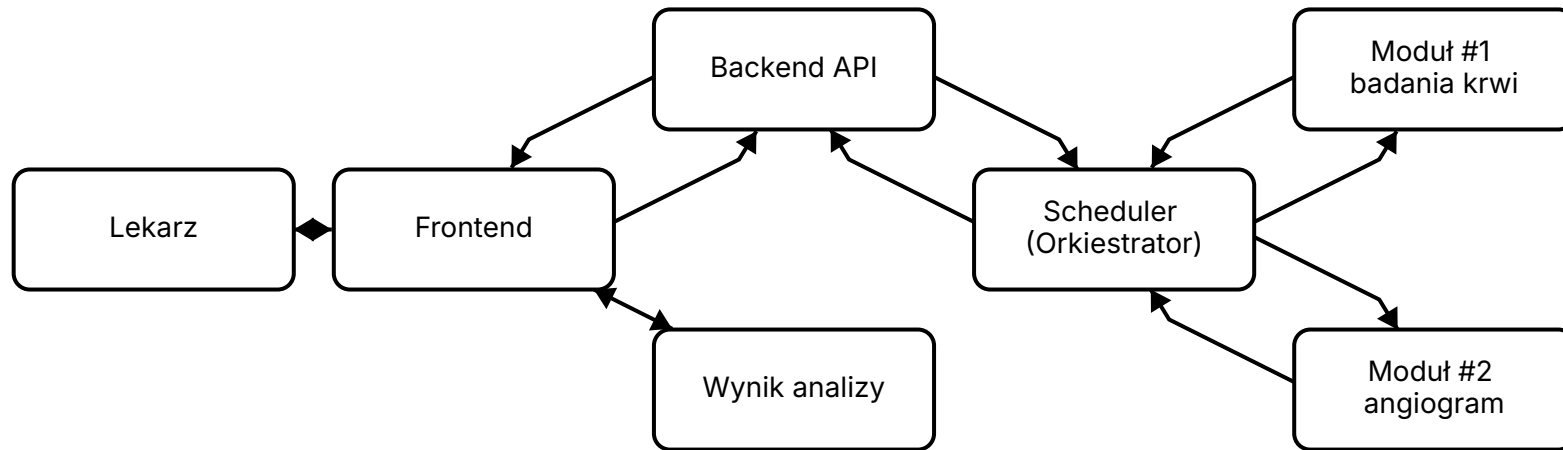
W ramach projektu wyróżniono dwa kluczowe procesy systemowe: proces wykonania analizy diagnostycznej oraz proces integracji nowego modułu diagnostycznego z platformą. Procesy te opisują główny sposób działania systemu oraz mechanizm jego dalszej rozbudowy.

2.4.1 Proces analizy diagnostycznej

Proces analizy diagnostycznej rozpoczyna się od przesłania przez lekarza anonimowych danych medycznych pacjenta oraz wskazania obszaru diagnostycznego, którego ma dotyczyć analiza. W przykładowym scenariuszu lekarz wybiera analizę w kierunku miażdżycy, natomiast platforma samodzielnie dobiera odpowiednie moduły diagnostyczne na podstawie typu dostarczonych danych.

Po otrzymaniu żądania backend przekazuje dane do schedulera, który pełni rolę orkiestratora modułów. Scheduler analizuje kontekst zapytania oraz typy danych wejściowych, a następnie kieruje żądanie do jednego lub wielu kompatybilnych modułów diagnostycznych. Wyniki zwrócone przez moduły są następnie zbierane przez platformę i prezentowane lekarzowi w formie czytelnej odpowiedzi.

Graf 2.2: Proces analizy diagnostycznej

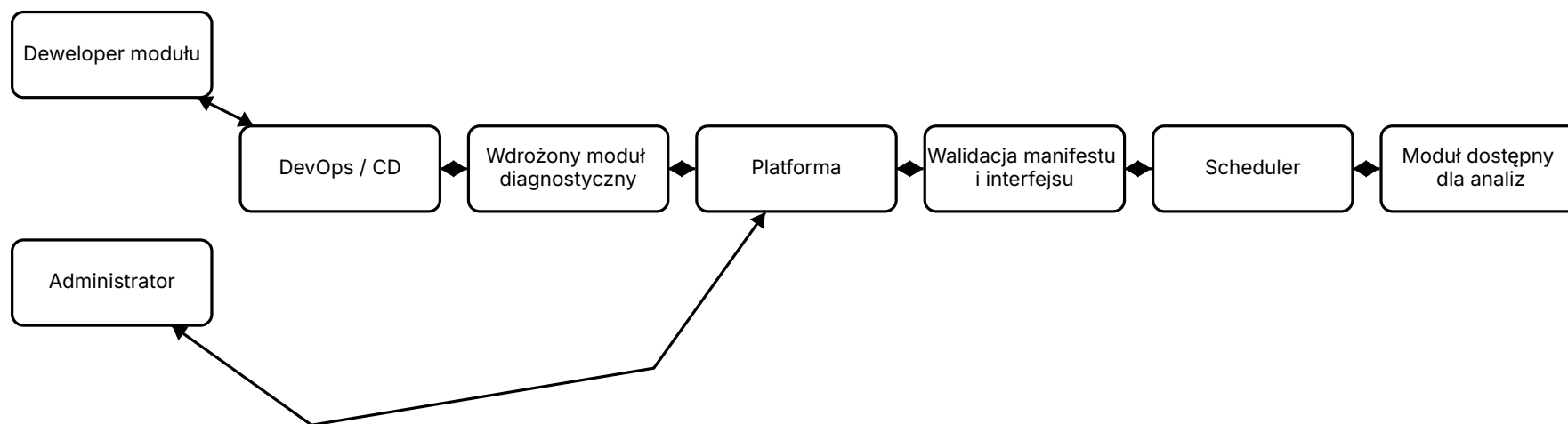


2.4.2 Proces integracji nowego modułu diagnostycznego

Proces integracji nowego modułu diagnostycznego zakłada, że sam moduł został wcześniej przygotowany przez dewelopera modułów oraz wdrożony w środowisku uruchomieniowym przez proces DevOps/CD. Moduł powinien być zgodny ze specyfikacją platformy oraz udostępniać wymagany interfejs komunikacyjny.

Administrator rejestruje moduł w platformie poprzez dostarczenie manifestu konfiguracyjnego lub wskazanie adresu modułu, z którego platforma może pobrać wymagane metadane. Następnie platforma waliduje informacje o module, takie jak obsługiwane typy danych, zakres diagnostyczny, format wejścia i wyjścia oraz endpointy komunikacyjne. Po pozytywnej walidacji moduł zostaje dodany do listy dostępnych modułów diagnostycznych i może być wykorzystywany przez scheduler podczas realizacji analiz.

Graf 2.3: Proces integracji nowego modułu diagnostycznego



2.5 Scenariusze użytkowania i testowania

W tej sekcji przedstawiono kluczowe scenariusze użycia systemu oraz sposób ich weryfikacji. Scenariusze opisują główne interakcje pomiędzy użytkownikami, platformą oraz modułami diagnostycznymi. Ze względu na różnorodność możliwych formatów danych medycznych, dokładne formaty wejściowe mogą zależeć od sposobu eksportu danych w danej placówce medycznej.

2.5.1 Przesłanie danych pacjenta do analizy

Aktor: Lekarz

Warunki początkowe:

- Lekarz posiada aktywne konto w systemie.
- Lekarz jest zalogowany do platformy.
- W systemie dostępny jest co najmniej jeden moduł diagnostyczny obsługujący wybrany typ analizy.

Przebieg scenariusza:

1. Lekarz przechodzi do formularza nowej analizy.
2. Lekarz wybiera obszar diagnostyczny, np. analizę w kierunku miażdżycy.
3. Lekarz przesyła anonimowe dane diagnostyczne pacjenta.
4. System waliduje poprawność przekazanych danych.
5. System przekazuje dane do dalszej analizy.
6. Lekarz otrzymuje wynik analizy w interfejsie użytkownika.

Rezultat: System prezentuje lekarzowi wynik zawierający sugestię diagnostyczną, informacje o poziomie pewności wyniku oraz listę modułów wykorzystanych w analizie.

Kryteria akceptacyjne:

- System umożliwia przesłanie danych diagnostycznych.
- System informuje użytkownika o błędnym lub nieobsługiwanym formacie danych.
- System prezentuje wynik analizy w czytelnej formie.
- System nie wymaga tworzenia trwałego profilu pacjenta.

2.5.2 Analiza danych przez moduł diagnostyczny

Aktor: Platforma / Moduł diagnostyczny

Warunki początkowe:

- Moduł diagnostyczny jest zarejestrowany w platformie.
- Moduł przeszedł walidację manifestu oraz podstawowy test dostępności.
- Scheduler posiada informacje o zakresie działania modułu.

Przebieg scenariusza:

1. Backend przekazuje żądanie analizy do schedulera.
2. Scheduler określa typ danych wejściowych oraz wybrany obszar diagnostyczny.
3. Scheduler wybiera moduły diagnostyczne zgodne z zakresem analizy.
4. Scheduler przekazuje dane do odpowiednich modułów.
5. Moduły wykonują analizę i zwracają wyniki.
6. Platforma agreguje wyniki i przygotowuje odpowiedź dla użytkownika.

Rezultat: Platforma zwraca ujednolicony wynik analizy niezależnie od liczby modułów wykorzystanych w procesie.

Kryteria akceptacyjne:

- Scheduler wybiera moduł zgodny z typem danych i celem analizy.
- Moduł zwraca wynik w formacie zgodnym ze specyfikacją platformy.
- System poprawnie obsługuje sytuację, w której żaden moduł nie obsługuje przekazanych danych.
- System obsługuje błędy komunikacji z modułem, np. timeout lub brak odpowiedzi.

2.5.3 Dodanie nowego modułu diagnostycznego

Aktor: Administrator

Warunki początkowe:

- Moduł diagnostyczny został wcześniej przygotowany i wdrożony jako niezależny mikroserwis.
- Moduł udostępnia interfejs zgodny ze specyfikacją platformy.
- Administrator posiada uprawnienia do zarządzania modułami.

Przebieg scenariusza:

1. Administrator przechodzi do panelu zarządzania modułami.
2. Administrator wskazuje adres modułu lub przesyła manifest konfiguracyjny.
3. System pobiera lub odczytuje metadane modułu.
4. System waliduje manifest oraz dostępność modułu.
5. System wykonuje podstawowy healthcheck modułu.
6. Po pozytywnej walidacji moduł zostaje dodany do listy dostępnych modułów.
7. Scheduler może wykorzystywać moduł podczas kolejnych analiz.

Rezultat: Nowy moduł diagnostyczny zostaje zarejestrowany w platformie i może być używany w procesie analizy danych.

Kryteria akceptacyjne:

- System umożliwia rejestrację modułu na podstawie manifestu lub adresu modułu.
- System weryfikuje poprawność wymaganych metadanych.
- System wykonuje podstawowy test dostępności modułu.
- Niepoprawny moduł nie zostaje dodany do platformy.
- Poprawnie dodany moduł jest widoczny dla schedulera.

2.5.4 Scenariusze testowe

2.5.4.1 Test przesyłania danych diagnostycznych

Celem testu jest sprawdzenie, czy lekarz może przesłać dane diagnostyczne i otrzymać odpowiedź z systemu.

Kroki testowe:

1. Zalogować się jako lekarz.
2. Wybrać obszar analizy.
3. Przesłać przykładowe dane diagnostyczne.
4. Uruchomić analizę.
5. Sprawdzić, czy wynik został wyświetlony.

Oczekiwany rezultat: System przyjmuje poprawne dane, uruchamia analizę oraz prezentuje wynik użytkownikowi.

2.5.4.2 Test wyboru modułu przez scheduler

Celem testu jest sprawdzenie, czy scheduler poprawnie wybiera moduł diagnostyczny na podstawie typu danych i obszaru analizy.

Kroki testowe:

1. Zarejestrować co najmniej dwa moduły diagnostyczne.
2. Przesłać dane obsługiwane tylko przez jeden z nich.
3. Uruchomić analizę.
4. Zweryfikować, który moduł został użyty.

Oczekiwany rezultat: Scheduler kieruje żądanie do modułu zgodnego z danymi wejściowymi i zakresem analizy.

2.5.4.3 Test rejestracji modułu diagnostycznego

Celem testu jest sprawdzenie poprawności procesu dodawania nowego modułu do platformy.

Kroki testowe:

1. Zalogować się jako administrator.
2. Wskazać adres modułu lub przesłać manifest.
3. Uruchomić proces rejestracji.
4. Sprawdzić wynik walidacji manifestu i healthchecku.
5. Zweryfikować, czy moduł pojawił się na liście dostępnych modułów.

Oczekiwany rezultat: Poprawny moduł zostaje zarejestrowany, a niepoprawny moduł zostaje odrzucony z czytelnym komunikatem błędu.

2.5.4.4 Test odporności na niedostępność modułu

Celem testu jest sprawdzenie, czy platforma poprawnie reaguje na brak odpowiedzi ze strony modułu diagnostycznego.

Kroki testowe:

1. Zarejestrować moduł diagnostyczny.
2. Zasymulować niedostępność modułu.
3. Uruchomić analizę wymagającą użycia tego modułu.
4. Sprawdzić reakcję systemu.

Oczekiwany rezultat: System nie kończy działania niekontrolowanym błędem, informuje o problemie z modułem oraz zapisuje zdarzenie w logach administracyjnych.

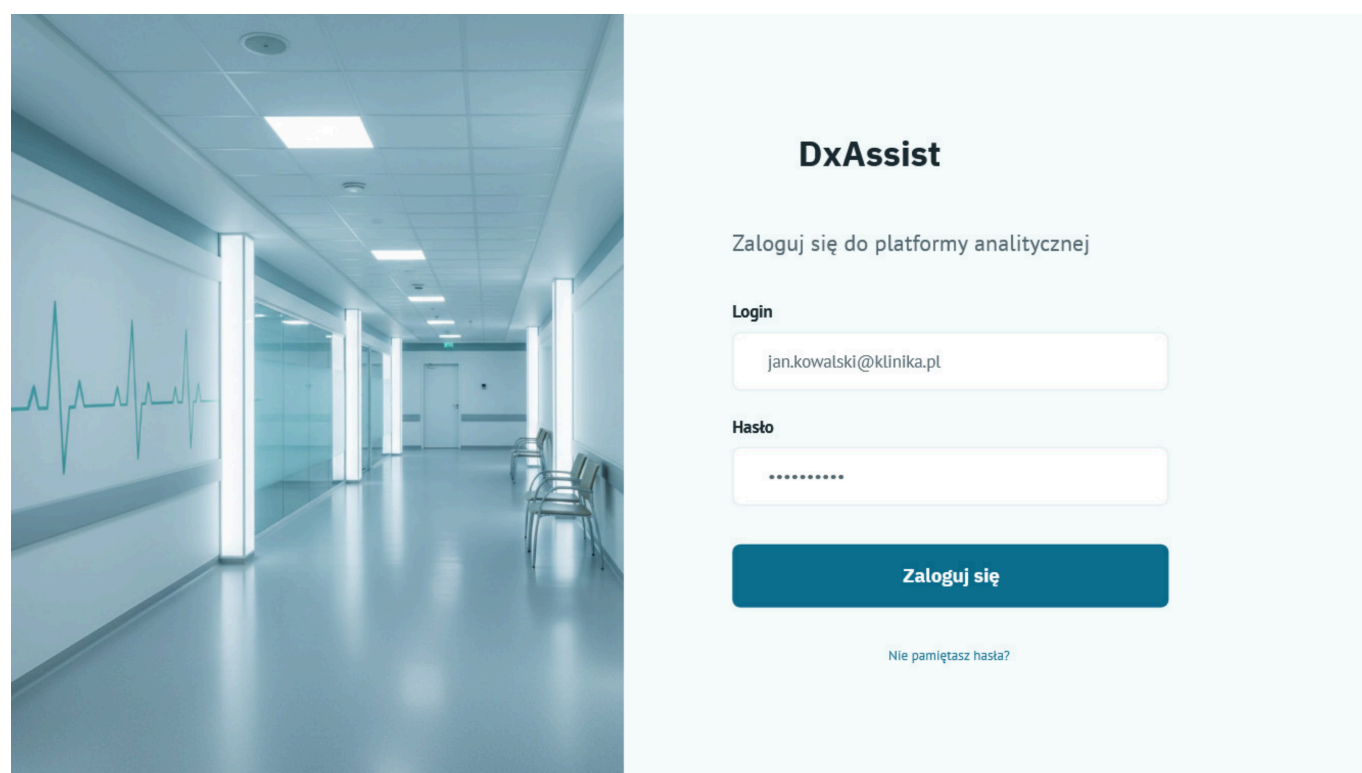
2.6 Koncepcja interfejsu użytkownika

Interfejs użytkownika projektowanego systemu został zaprojektowany z myślą o prostocie obsługi oraz czytelnej prezentacji wyników diagnostycznych. Ze względu na docelową grupę użytkowników końcowych szczególny nacisk położono na intuicyjność interfejsu oraz ograniczenie liczby zbędnych operacji wymaganych do wykonania analizy.

Platforma przewiduje rozdzielenie interfejsów zgodnie z rolami użytkowników. Lekarze korzystają głównie z funkcji związanych z przysyłaniem danych diagnostycznych oraz przeglądaniem wyników analiz, natomiast administratorzy posiadają dostęp do funkcji zarządzania użytkownikami i modułami diagnostycznymi.

2.6.1 Ekran logowania

Ekran logowania umożliwia użytkownikowi uwierzytelnienie w systemie przy użyciu loginu oraz hasła. Po poprawnym zalogowaniu użytkownik uzyskuje dostęp do funkcji zgodnych z przypisaną rolą systemową.



Makieta 2.1: Przykładowa strona logowania

2.6.2 Panel lekarza

Panel lekarza stanowi główny interfejs roboczy systemu. Umożliwia wybór obszaru diagnostycznego, przesyłanie danych medycznych oraz uruchamianie procesu analizy. Interfejs powinien prezentować dostępne typy analiz w uproszczonej i czytelnej formie.

W ramach panelu lekarza przewidziano możliwość przesyłania różnych typów danych diagnostycznych, w tym dokumentów tekstowych, wyników badań laboratoryjnych oraz obrazów medycznych.

DxAssist

Przesyłanie danych

Krok 1 z 2

Plik z danymi medycznymi

Przeciągnij pliki tutaj

lub

Wybierz plik

Obsługiwane formaty: CSV, XLSX, DICOM, HL7

Choroby do analizy

- ☒ Nadciśnienie
- ☒ Miażdżycy
- ☐ Zawał
- ☐ Niewydolność Nerek

Analizuj

Makieta 2.2: Przykładowy panel lekarza

2.6.3 Formularz przesyłania badań

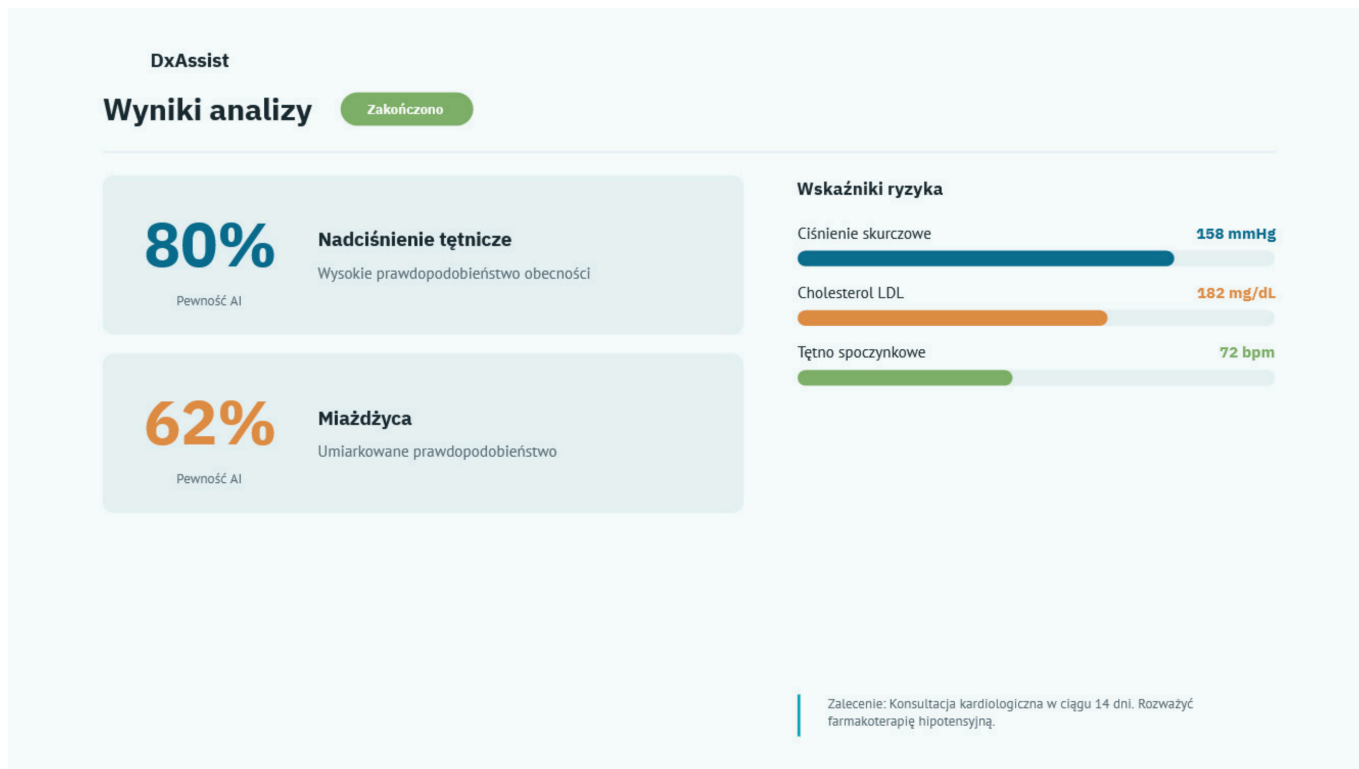
Formularz przesyłania danych umożliwia przekazanie plików diagnostycznych do platformy. Interfejs powinien obsługiwać różne formaty danych wejściowych oraz informować użytkownika o poprawności przesłanych danych.

W formularzu przewidziano możliwość dodania jednego lub wielu plików oraz określenia rodzaju wykonywanej analizy.

2.6.4 Widok wyników analizy

Widok wyników prezentuje odpowiedź wygenerowaną przez moduły diagnostyczne w czytelnej formie. Wynik może zawierać sugestię diagnostyczną, poziom pewności predykcji, wykorzystane moduły diagnostyczne oraz dodatkowe informacje pomocnicze.

Interfejs powinien umożliwiać szybkie odczytanie najważniejszych informacji bez konieczności analizy technicznych szczegółów działania modeli.



Makieta 2.3: Przykładowy ekran wyników analizy

2.6.5 Panel administratora

Panel administratora umożliwia zarządzanie użytkownikami oraz modułami diagnostycznymi zarejestrowanymi w platformie. Administrator posiada możliwość dodawania nowych modułów, monitorowania ich dostępności oraz kontrolowania podstawowych parametrów działania systemu. Interfejs administratora powinien umożliwiać szybki podgląd aktualnie dostępnych modułów oraz ich statusu.

2.6.6 Założenia projektowe interfejsu

Projekt interfejsu użytkownika opiera się na następujących założeniach:

- prostota oraz czytelność prezentowanych informacji,
- ograniczenie liczby operacji wymaganych do wykonania analizy,
- dostosowanie interfejsu do użytkowników nietechnicznych,
- czytelna prezentacja wyników modeli uczenia maszynowego,
- rozdzielenie funkcji zgodnie z rolami użytkowników,
- możliwość dalszej rozbudowy interfejsu wraz z rozwojem platformy.

2.7 Specyfikacja API i komunikacji

Komunikacja pomiędzy frontendem a backendem platformy będzie realizowana w formie API typu REST. Komunikacja wewnętrzna pomiędzy backendem, schedulerem oraz modułami diagnostycznymi może wykorzystywać REST lub inne mechanizmy komunikacyjne, zależnie od szczegółowych decyzji implementacyjnych. Na obecnym etapie zakłada się asynchroniczny model wykonywania analiz, w którym zgłoszenie analizy tworzy zadanie, a wynik jest pobierany po zakończeniu przetwarzania.

2.7.1 API backendu

API backendu odpowiada za obsługę użytkowników, uwierzytelnianie, przyjmowanie danych diagnostycznych, uruchamianie analiz oraz prezentację wyników. Endpointy przedstawione poniżej mają charakter projektowy i mogą ulec zmianie podczas implementacji.

Metoda	Endpoint	Opis
POST	/api/auth/login	Uwierzytelnienie użytkownika i rozpoczęcie sesji lub wydanie tokenu dostępowego.
POST	/api/auth/logout	Zakończenie sesji użytkownika lub unieważnienie tokenu.
GET	/api/users	Pobranie listy użytkowników systemu przez administratora.
POST	/api/users	Utworzenie nowego konta użytkownika przez administratora.
POST	/api/analyses	Utworzenie nowego zadania analizy diagnostycznej na podstawie przesłanych danych medycznych oraz wybranego obszaru diagnostycznego.
GET	/api/analyses/{id}	Pobranie statusu zadania analizy oraz wyniku, jeśli analiza została zakończona.
GET	/api/modules	Pobranie listy dostępnych modułów diagnostycznych.
POST	/api/modules	Rejestracja nowego modułu diagnostycznego na podstawie manifestu lub adresu modułu.

Przesyłanie danych diagnostycznych będzie realizowane jako ogólny mechanizm przyjmowania plików oraz metadanych analizy. Ze względu na różnorodność formatów danych medycznych dokładna walidacja obsługiwanych plików może zostać wykonana dopiero po stronie schedulera lub modułów diagnostycznych.

Przykładowe zgłoszenie analizy może zawierać dane w formacie `multipart/form-data`, obejmujące:

- plik lub zestaw plików diagnostycznych,
- wybrany obszar analizy, np. *atherosclerosis*,
- opcjonalne metadane opisujące typ badania.

2.7.2 Komunikacja backendu z schedulerem

Backend przekazuje do schedulera informacje o zadaniu analizy oraz lokalizacji przesłanych danych. Scheduler odpowiada za wybór odpowiednich modułów diagnostycznych i koordynację procesu analizy.

Element	Opis
Wejście	Identyfikator zadania analizy, typ analizy, metadane danych wejściowych oraz lokalizacja plików lub danych przekazanych przez użytkownika.
Wyjście	Status przetwarzania, lista wykorzystanych modułów oraz zagregowany wynik analizy.

Element	Opis
Charakter komunikacji	Asynchroniczny. Backend tworzy zadanie analizy, a wynik jest pobierany po zakończeniu przetwarzania.
Możliwa implementacja	REST API, komunikacja zdarzeniowa lub WebSocket, zależnie od decyzji implementacyjnych.

2.7.3 API modułów diagnostycznych

Każdy moduł diagnostyczny powinien udostępniać minimalny interfejs umożliwiający platformie sprawdzenie jego dostępności, pobranie metadanych oraz uruchomienie analizy. Szczegółowy format tego interfejsu zostanie doprecyzowany w specyfikacji integracji modułów.

Metoda	Endpoint	Opis
GET	/health	Sprawdzenie dostępności modułu oraz podstawowego stanu działania usługi.
GET	/metadata	Pobranie informacji o module, obsługiwanych typach danych, zakresie diagnostycznym oraz formacie wejścia i wyjścia.
POST	/analize	Uruchomienie analizy danych przekazanych przez scheduler.

Moduł diagnostyczny powinien zwracać wyniki w ujednoliconym formacie umożliwiającym ich dalszą agregację przez platformę. Wynik może zawierać sugestię diagnostyczną, wynik liczbowy, poziom pewności oraz dodatkowe informacje pomocnicze.

2.7.4 Manifest modułu diagnostycznego

Rejestracja modułu diagnostycznego może odbywać się poprzez dostarczenie manifestu konfiguracyjnego lub wskazanie adresu modułu, z którego platforma pobierze wymagane metadane. Manifest opisuje podstawowe informacje potrzebne do integracji modułu z platformą.

Przykładowy manifest modułu:

```
# Unique module identifier
module_id: blood-screening-atherosclerosis

# Human-readable module name
name: Blood Screening Atherosclerosis Module

# Short module description
description: Module for analyzing blood screening results in the context of atherosclerosis risk.

# Diagnostic domain supported by the module
diagnostic_area: atherosclerosis

# Supported input data types
input_types:
  - blood_test
  - laboratory_results

# Supported input formats
input_formats:
  - json
  - csv

# Module API base URL
base_url: http://blood-module.local:8000

# Required API endpoints
endpoints:
  health: /health
  metadata: /metadata
  analyze: /analyze

# Output format returned by the module
output_format:
  type: json
  fields:
    - risk_score
    - confidence
    - recommendation
    - details

# Optional resource information
resources:
  cpu: "1"
  memory: "512Mi"
```

2.7.5 Format danych

Platforma powinna umożliwiać obsługę różnych typów danych diagnostycznych, ponieważ formaty wejściowe mogą zależeć od sposobu ich eksportu w konkretnej placówce medycznej. W przypadku danych laboratoryjnych możliwe jest wykorzystanie formatów strukturalnych, takich jak JSON lub CSV, natomiast dane obrazowe mogą być dostarczane w formatach takich jak DICOM, TIFF, PNG lub JPEG.

Przykładowy uproszczony format danych wejściowych dla badań laboratoryjnych:

```
{
  "analysis_type": "atherosclerosis",
  "input_type": "blood_test",
  "data": {
    "total_cholesterol": 220,
    "ldl": 145,
    "hdl": 42,
    "triglycerides": 180
  }
}
```

Przykładowy format wyniku analizy:

```
{
  "analysis_id": "analysis-123",
  "status": "completed",
  "diagnostic_area": "atherosclerosis",
  "used_modules": [
    "blood-screening-atherosclerosis"
  ],
  "result": {
    "risk_score": 0.72,
    "confidence": 0.81,
    "recommendation": "Wynik sugeruje podwyższone ryzyko zmian miażdżycowych. Wynik powinien zostać zweryfikowany przez lekarza.",
    "details": {
      "important_factors": [
        "ldl",
        "total_cholesterol",
        "triglycerides"
      ]
    }
  }
}
```

Podane struktury mają charakter projektowy i służą jako punkt wyjścia do dalszego doprecyzowania kontraktów komunikacyjnych pomiędzy komponentami systemu.

Bibliografia

- [1] K. Saab *et al.*, „Advancing Conversational Diagnostic AI with Multimodal Reasoning”. [Online]. Dostępne na: <https://arxiv.org/abs/2505.04653>
- [2] L. Yang *et al.*, „Advancing Multimodal Medical Capabilities of Gemini”. [Online]. Dostępne na: <https://arxiv.org/pdf/2405.03162>
- [3] K. Saab *et al.*, „Capabilities of Gemini Models in Medicine”. [Online]. Dostępne na: <https://arxiv.org/pdf/2404.18416>